

Resolución del Práctico 7

Testing de Software 2026

Tomás Rodeghiero

Índice

1. Ejercicio 1: NodeCachingLinkedList	2
1.1. <code>repOK()</code>	2
1.2. Bug detectado: inicialización de <code>maximumCacheSize</code>	2
1.3. Tres propiedades con <code>jqwik</code>	3
2. Ejercicio 2: Point	4
2.1. Relación entre <code>equals</code> y <code>hashCode</code>	4
2.2. Implementación	4
2.3. Generador y propiedad del contrato	5
2.4. Propiedad geométrica: recta paralela al eje X	5
3. Ejercicio 3: Date	7
3.1. Constructor y <code>repOk()</code>	7
3.2. <code>addDays(Date when, int days)</code>	8
3.3. Propiedad con generadores	8

1. Ejercicio 1: NodeCachingLinkedList

`NodeCachingLinkedList` (paquete `assignment7_exercises.ncl`) es una lista doblemente enlazada y circular, con un *pool* interno de nodos reutilizables (la *cache*) para evitar crear y descartar objetos todo el tiempo. La consigna pide completar `repOK()` y escribir tres propiedades con `jqwik`.

1.1. `repOK()`

La estructura tiene dos “mitades” con reglas distintas: la **lista principal** (circular, doblemente enlazada, con centinela `header`) y la **cache** (lista simplemente enlazada de nodos reutilizables). El chequeo arranca por las condiciones baratas y avanza hacia las más costosas:

- Validez básica del `header`: no puede ser `null` y sus enlaces `next/previous` tampoco.
- Consistencia de tamaños: `size`, `cacheSize` y `maximumCacheSize` no negativos, y `cacheSize <= maximumCacheSize`.
- Constante `DEFAULT_MAXIMUM_CACHE_SIZE == 20` (el enunciado pide verificarla explícitamente).
- Recorrido de la lista principal: avanzar desde `header` siguiendo `next` hasta volver al `header`. En cada paso, los enlaces dobles tienen que ser coherentes (`node.next.previous == node` y `node.previous.next == node`). Uso un `HashSet` para detectar ciclos prematuros.
- Coherencia de `size`: al terminar, la cantidad de nodos visitados menos uno (el centinela) tiene que coincidir con `size`.
- Recorrido de la cache: avanzar desde `firstCachedNode` siguiendo `next` hasta `null`. En cada nodo: `previous == null` y `value == null` (estado “limpio”), sin ciclos, y sin pertenecer a la lista principal.
- Coherencia de `cacheSize`: la cantidad de nodos recorridos en la cache tiene que coincidir con `cacheSize`.

1.2. Bug detectado: inicialización de `maximumCacheSize`

El constructor original dejaba `maximumCacheSize = 0`, lo que en la práctica deshabilita la cache (si `cacheSize >= maximumCacheSize` siempre es cierto, nunca se cachean nodos). Eso va en contra de la idea misma de la estructura. La corrección fue inicializar `maximumCacheSize = DEFAULT_MAXIMUM_CACHE_SIZE` en el constructor:

```

1 public NodeCachingLinkedList() {
2     header = new LinkedListNode();
3     header.setValue(null);
4     header.setNext(header);
5     header.setPrevious(header);
6     maximumCacheSize = DEFAULT_MAXIMUM_CACHE_SIZE;
7 }

```

1.3. Tres propiedades con jqwik

Propiedad 1: al remover un elemento, la cache crece en uno

```

1 @Property(tries = 200)
2 void luegoDeRemoverUnElementoSeIncrementaEnUnoElTamanoDeCache(
3     @ForAll("escenariosRemocion") EscenarioRemocion escenario
4 ) {
5     NodeCachingLinkedList ncl = escenario.lista;
6     assertTrue(ncl.repOK());
7     int cacheAntes = ncl.getCacheSize();
8
9     Integer removido = ncl.removeIndex(escenario.index);
10
11     assertNotNull(removido);
12     assertEquals(cacheAntes + 1, ncl.getCacheSize());
13 }

```

El generador arma una lista con entre 1 y 15 elementos y un índice válido. La propiedad se cumple mientras no se supere `maximumCacheSize`, condición garantizada por el rango de tamaños ($\leq 15 < 20$).

Propiedad 2: con cache no vacía, agregar conserva la suma total de nodos

Cuando la cache tiene nodos, un `addFirst` no crea un nodo nuevo: reutiliza uno de la cache. Por lo tanto `size + cacheSize` queda constante. El generador primero arma una lista y después remueve un elemento (para que la cache tenga al menos un nodo); recién ahí ejecuta el `addFirst` y compara la suma antes y después.

Propiedad 3: remover un elemento preserva `repOK()`

Una de las propiedades clásicas de los invariantes: cualquier operación pública sobre un objeto válido debe dejarlo en un estado válido. Si alguna operación interna (manejo de en-

laces, paso a cache, decremento de `size`) dejara la estructura inconsistente, esta propiedad lo detectaría.

Las tres propiedades corren con `tries = 200`.

2. Ejercicio 2: Point

Trabajo sobre `assignment7_exercises.point.Point`, que modela un punto en el plano con coordenadas `x` e `y` de tipo `float`.

2.1. Relación entre `equals` y `hashCode`

El contrato de `Object` establece dos reglas que son clave cuando una clase se usa en `HashMap` o `HashSet`:

- Si dos objetos son iguales según `equals`, deben tener el mismo `hashCode`. Es obligatorio: si se rompe, las estructuras basadas en hashing dejan de funcionar.
- La inversa **no** es obligatoria. Dos objetos con el mismo hash no tienen por qué ser iguales: eso es una *colisión*, esperable porque el hash es un entero con rango finito.

Al sobrescribir `equals` también hay que sobrescribir `hashCode` de manera consistente.

2.2. Implementación

```
1 @Override
2 public boolean equals(Object obj) {
3     if (this == obj) return true;
4     if (obj == null || getClass() != obj.getClass()) return false
5     ;
6     Point other = (Point) obj;
7     return Float.compare(this.x, other.x) == 0
8         && Float.compare(this.y, other.y) == 0;
9 }
10
11 @Override
12 public int hashCode() {
13     int result = Float.floatToIntBits(x);
14     result = 31 * result + Float.floatToIntBits(y);
15     return result;
16 }
```

Detalles importantes:

- `equals` usa `Float.compare(...)` en vez de `==`. `==` trata mal los casos de IEEE 754: `NaN == NaN` da `false` (rompería la reflexividad si un punto tuviera `NaN`); `-0.0f == +0.0f` da `true` aunque los bits sean distintos. `Float.compare` resuelve los dos casos.
- `hashCode` combina las coordenadas con `Float.floatToIntBits(...)`, que mapea el `float` a su representación binaria como `int`. Eso garantiza que dos `float` “iguales” según `Float.compare` produzcan el mismo `int` y, por lo tanto, el mismo hash.

2.3. Generador y propiedad del contrato

```

1 @Provide
2 Arbitrary<Point> puntos() {
3     return Combinators.combine(coordenadas(), coordenadas())
4         .as(Point::new);
5 }
6
7 @Provide
8 Arbitrary<Float> coordenadas() {
9     return Arbitraries.floats().between(-10_000f, 10_000f);
10 }
11
12 @Property(tries = 250)
13 void puntosIgualesDebenTenerMismoHashCode(@ForAll("puntos") Point
14     punto) {
15     Point mismoPunto = new Point(punto.getX(), punto.getY());
16     assertTrue(punto.equals(mismoPunto));
17     assertEquals(punto.hashCode(), mismoPunto.hashCode());
18 }

```

El rango `[-10_000, 10_000]` se acotó a propósito: evita `Float.NaN` y `Float.POSITIVE_INFINITY`, que generarían ruido sin agregar valor.

2.4. Propiedad geométrica: recta paralela al eje X

Cuando dos puntos comparten la ordenada y , están sobre una recta horizontal y su distancia se reduce a $|x_2 - x_1|$. Buena propiedad para PBT: captura una invariante geométrica real, independiente de los valores concretos.

```

1 @Property(tries = 250)
2 void distanciaEnRectaParalelaAlEjeXEsDiferenciaDeAbscisas(
3     @ForAll("coordenadas") float x1,
4     @ForAll("coordenadas") float x2,
5     @ForAll("coordenadas") float y

```

```
6 ) {  
7     Point p1 = new Point(x1, y);  
8     Point p2 = new Point(x2, y);  
9  
10    double distanciaEsperada = Math.abs((double) (x2 - x1));  
11    double distanciaReal = p1.distanceTo(p2);  
12  
13    assertEquals(distanciaEsperada, distanciaReal, 1.0e-6);  
14 }
```

La tolerancia $1e-6$ es necesaria porque `distanceTo` implementa la fórmula euclídea con `Math.sqrt(Math.pow(...))` y esas operaciones introducen errores de redondeo inevitables. Sin tolerancia, la propiedad fallaría por diferencias ínfimas en el último bit.

3. Ejercicio 3: Date

Trabajo sobre `assignment7_exercises.date.Date`, que representa una fecha de calendario (día, mes, año) desde 1900.

3.1. Constructor y `repOk()`

El constructor valida la terna con `isValidDate` antes de asignar. Si la combinación no es legal, lanza `IllegalArgumentException("invalid date")`. Así nunca se construye un `Date` en estado inválido.

```
1 public Date(int d, int m, int y) throws IllegalArgumentException
   {
2     if (!isValidDate(d, m, y)) {
3         throw new IllegalArgumentException("invalid date");
4     }
5     this.day = d;
6     this.month = m;
7     this.year = y;
8     assert repOk();
9 }
10
11 public boolean repOk() {
12     return isValidDate(day, month, year);
13 }
```

Reglas del invariante (centralizadas en `isValidDate`):

- `year >= 1900`.
- `1 <= month <= 12`.
- `1 <= day <= daysInMonth(month, year)`.
- Meses de 30 días (4, 6, 9, 11) no admiten `day == 31`.
- Febrero tiene 29 días en años bisiestos y 28 en años no bisiestos.

`repOk()` reutiliza `isValidDate` para no duplicar lógica. El cálculo de días por mes está centralizado en `daysInMonth(m, y)`, que también consume `addDays`: tener un único lugar evita desincronizaciones.

3.2. addDays(Date when, int days)

Devuelve una **nueva** Date (no muta la original), sumando **days** días calendario a **when**. Avanza por bloques de mes en lugar de día por día: con **days** del orden de miles, la iteración por día sería innecesariamente lenta.

```
1 public Date addDays(Date when, int days) {
2     if (when == null) throw new IllegalArgumentException("date
3         cannot be null");
4
5     if (days < 0) throw new IllegalArgumentException("days
6         must be non-negative");
7
8     int d = when.day, m = when.month, y = when.year;
9     int remainingDays = days;
10
11     while (remainingDays > 0) {
12         int daysInCurrentMonth = daysInMonth(m, y);
13         int daysLeftInMonth = daysInCurrentMonth - d;
14
15         if (remainingDays <= daysLeftInMonth) {
16             d = d + remainingDays;
17             remainingDays = 0;
18         } else {
19             remainingDays = remainingDays - (daysLeftInMonth + 1)
20                 ;
21             d = 1;
22             m = m + 1;
23             if (m > 12) { m = 1; y = y + 1; }
24         }
25     }
26
27     return new Date(d, m, y);
28 }
```

La clave: nunca se construye una fecha intermedia inválida. El salto de mes se hace en un solo paso, sin pasar por un hipotético “32 de enero”. Y el `new Date(d, m, y)` final vuelve a pasar por el constructor, así que cualquier estado inválido explotaría con excepción.

3.3. Propiedad con generadores

```
1 @Property(tries = 250)
2 void addDaysSiempreDevuelveFechaValida(
3     @ForAll("fechasValidas") Date fecha,
```



```

4      @ForAll("diasNoNegativos") int dias
5  ) {
6      Date receptor = new Date(1, 1, 1900);
7      Date resultado = receptor.addDays(fecha, dias);
8
9      assertNotNull(resultado);
10     assertTrue(resultado.repOk());
11 }
12
13 @Provide
14 Arbitrary<Date> fechasValidas() {
15     return Arbitraries.integers().between(1900, 2400).flatMap(
16         year ->
17             Arbitraries.integers().between(1, 12).flatMap(month ->
18                 Arbitraries.integers().between(1, daysInMonth(month,
19                     year)).map(day ->
20                         new Date(day, month, year)
21                     )
22             )
23     );
24 }
25
26 @Provide
27 Arbitrary<Integer> diasNoNegativos() {
28     return Arbitraries.integers().between(0, 5000);
29 }

```

El generador es lo más interesante: el rango del día depende del mes y el año ya elegidos, por eso se usa `flatMap` anidado. Si se generaran `day`, `month`, `year` de forma independiente, aparecerían combinaciones como 31/04/2024 que el constructor rechazaría y la propiedad nunca llegaría a evaluarse.

El rango de días (0 a 5000) está pensado para forzar saltos de varios años (5000 días $\approx$ 13 años y medio) sin volver lenta la suite.